

Resilience

A hands-on course in building agents by building their harness

Lesson 8

Everything so far assumed the other side answers

Over a long run, this is what breaks first:

- the API drops a call, or the model is overloaded
- a command fails because something else had a file locked
- a response is cut off in the middle of a command
- the process dies halfway through, and the work goes with it
- or you stop it with ESC, and what it had already done goes with it

Built on the model interface and output

- The **model interface** is a service across a network: slow, busy or gone
- **Output** runs the model's words: a command can fail, a request can arrive broken
- Both are where work is in flight when you press ESC
- To pick a run back up, it **reads context's own record**: Lesson 4's episode

A production layer: hardening, not a new primitive.

Not every failure means the same thing

Worth trying again	Not worth trying again
the network dropped, or a timeout	a bad API key (401)
"slow down" (429)	a request the API won't accept (400)
any 5xx, and "overloaded" (529)	a model that doesn't exist (404)

The SDK already retries the left column, with backoff and jitter (`max_retries`).
What it can't do is choose a different model.

When you press ESC, keep what was in flight

Lesson 6 stops at once by throwing the partials away. But the words were written, and the command did its first steps in the world.

- What it had said goes into working memory in **whole blocks**
- Text that had started, and thinking that had finished and been **signed**
- A tool request it had begun: [your doing never reached the world]
- A stopped command keeps what it printed, then [your doing stopped before done]

The concept: a call with somewhere else to go

resilience.py , call() (abridged):

```
client = Anthropic(max_retries=2)
MODELS = ["claude-sonnet-5-5", "claude-opus-5-5"]

def call(input):
    for model in MODELS:
        try:
            response = client.messages.create(model=model, ...)
            return f"[{model}] " + ...
        except (APIConnectionError, APIStatusError) as e:
            if ... e.status_code < 500 and e.status_code != 429: raise
            print(f"[{model}: {type(e).__name__}, after the SDK's retries]")
    return "[no model answered]"
```

The concept: a failed command is reported

`run()`, the safe default (abridged):

```
def run(cmd):  
    done = subprocess.run(cmd, shell=True, ...)  
    print(done.stdout, end="")  
    if not done.returncode: return "[done]"  
    return f"[exit {done.returncode}: failed]"
```

A locked database, and `migrate.py` failing at its fourth table:

```
OperationalError (SQLITE_BUSY): database is locked  
[exit 1: failed]
```

```
error: payments: no column named email  
[exit 1: failed]
```

With Jev: what kind of failure?

`jev_resilience.py` is `resilience.py` plus these lines in `run()` (abridged):

```
try: answers = jev.system_one({"command": cmd, "result": ...},
                             {"failure": FAILURE, "kind": KIND}).choices
except Exception: answers = None
if answers:
    why, kind = answers["failure"], answers["kind"]
    if attempt == 1 and sure(why, "transient") and sure(kind, "read"):
        print("[a failure that passes: trying once more]"); ...; return run(cmd, 2)
    if sure(why, "partial"): return f"[exit ...: failed] (it may have partly run: ...)"
```

- `FAILURE` : **transient**, **permanent** or **partial**; `KIND` : does it only read?
- Anything else, or no answer: the plain `[exit N: failed]`

With Jev: the same failures

The locked database, with the lock let go when Jev answers:

```
OperationalError (SQLITE_BUSY): database is locked  
[Jev: transient 1.00, read 0.94]  
[a failure that passes: trying once more]  
(42, )  
[done]
```

```
error: payments: no column named email  
[Jev: partial 0.92, change 1.00]  
[exit 1: failed] (it may have partly run: check before repeating it)
```

- A typo: `permanent 1.00` , no second try. No key: `[exit 1: failed]` , as before

quark: `call()` grows a backup

Lesson 7's `quark.py` plus 43 lines, 395 in all (abridged):

```
client = Anthropic(timeout=300, max_retries=3)
MODELS = ["claude-sonnet-5-5", "claude-opus-5-5"]
class Down(Exception): pass
def call(each=lambda event: None, **request):
    for model in MODELS:
        try:
            with client.messages.stream(model=model, **request) as stream:
                ...
        except (APIConnectionError, APIStatusError) as e:
            if ... e.status_code < 500 and e.status_code != 429: raise
            trace(event="model_failed", model=model, error=type(e).__name__)
    raise Down()
```

quark: Jev's question

After Lesson 5's `ask()` (abridged):

```
FAILURE = Choice(instructions="... What kind of failure is it?", criteria=...)
def failure(cmd, result):
    why = ask({"command": cmd, "result": result[-4000:]}, FAILURE)
    return why["choice"] if why and why["confidence"] >= SURE else None
def reads(cmd):
    kind = ask({"command": cmd}, KIND)
    return bool(kind and kind["choice"] == "read" and kind["confidence"] >= SURE)
```

- `failure()` : the kind of failure, only when Jev is sure
- `reads()` : Lesson 6's `KIND` again, yes only to a sure `read`

quark: one more try

The command now runs in a loop of at most two tries (abridged):

```
for attempt in (1, 2):
    ...
    trace(event="tool", ..., failed=failed(cmd, done.stdout))
    why = failure(cmd, done.stdout) if done.returncode and not ESC.is_set() else None
    if attempt == 2 or why != "transient" or not reads(cmd): break
    print("[a failure that passes: trying once more]"); time.sleep(1)
...
if why == "partial": done.stdout += "\n(it may have partly run: ...)"
```

Picking up an unfinished session

In # — input — (abridged):

```
resumed = None if sys.argv[1:] else unfinished()
```

- `unfinished()` reads the newest episode; if it ends mid-task, it asks: *pick it up?* *[y/N]*
- `EPISODE` points at the old file, so the session continues in it
- A request with no result gets: *"interrupted: ... it may or may not have run. Check before repeating it."*
- The harness doesn't run it again: it might have been `git push`

ESC, Down, and half a command

While it thinks or says, what it had produced is kept (abridged):

```
kept = [b for b in output if (b.type != "text" or b.text)
        and (b.type != "thinking" or b.signature)]
```

- While it acts, Lesson 6's `=` becomes `+=`: a stopped command keeps what it printed
- `except Down:` ends the run with a sentence: *run quark again to pick it up*
- No `cmd`, or cut off by `max_tokens`: `[cut off, not run]`, and the model is told it *never reached the world*

Run it: the preferred model is down

`flaky.py` answers "529 overloaded" to every request for `claude-sonnet-5-5`:

```
$ wc -c notes.txt
6 notes.txt

notes.txt is 6 bytes.
```

The output looks like any other run. The trace doesn't (shortened):

```
{"event": "model_failed", "model": "claude-sonnet-5-5", "error": "OverloadedError"}
{"event": "model", "seconds": 4.91, "stop_reason": "tool_use"}
```

Run it: nothing answers, then the API comes back

Every request fails, for every model:

```
[the model isn't answering. Everything so far is in the episode; run quark again to pick it up]
```

Run `quark.py` again with no input, and answer `y` to *unfinished session: 'How many bytes is notes.txt? Answer in one line.'. pick it up?* (shortened):

```
6 notes.txt
```

```
notes.txt is 6 bytes.
```

Run it: killed mid-command, then resumed

`sleep 15 && echo finished > flag.txt` , killed with `kill -9` on its PID eight seconds in. The container wasn't, so the command finished. Resumed (shortened):

```
$ ls -l flag.txt; cat flag.txt  
-rw-rw-rw- 1 root root 9 Oct  6 23:21 flag.txt  
finished
```

The harness said my first run was interrupted, so I checked the file before running the command again. The file already existed with that content, which means the command had finished. I didn't run it a second time.

Run it: a failure that passes

The locked `shop.db` , through quark:

```
$ python3 -m sqlite3 shop.db "select count(*) from orders"  
[a failure that passes: trying once more]  
(42,)
```

There are 42 orders in shop.db.

```
{"event": "tool", "seconds": 5.4, "exit": 1, "failed": 0.99}  
{"event": "tool", "seconds": 0.32, "exit": 0, "failed": 0.03}
```

Run it: ESC, and the model knows how far it got

While saying, the screen stopped at (shortened):

... Likewise $19 \times 52631 = 999,989$, which leaves

After ESC (shortened):

Understood, I'll stop there. ... The hand checks I did cover 2, 3, 5, 7, 11, 13, 17, 19 and 37, and none of them divides it.

While doing, a line a second, ESC after about three: the model saw `step 1` to `step 3` and `[your doing stopped before done]` .

Run it: every request cut off, nothing run

A copy with `max_tokens` lowered to 400, and a 40-line poem to write in one command (shortened):

```
$ None  
[cut off, not run]  
$ None  
[cut off, not run]
```

- Each request arrived with no `cmd` at all, so the line says `$ None`
- Each was answered with the cut-off result; none of them ran
- After the tenth, the model stopped and said what was happening, with nothing half-written on disk

Other things we could do

- **Waiting and the backup:** backoff with a cap, `Retry-After`, a limiter; another model, provider or region
- **Remembering a failure:** a *bench* or *circuit breaker* leaves a failed model alone for a minute
- **Commands:** kill the whole process group, cap output, a hand-written list of reads
- **State:** a checkpoint file written whole or not at all; restart by hand or by a supervisor
- **Products:** gateways (LiteLLM, OpenRouter); durable execution (Temporal), checkpointing (LangGraph)

The rule

Assume the call can fail, and make failure something the harness handles instead of something that ends the run.

- Retry what passes, a bounded number of times; have somewhere else to go
- Run a failed command again only when it's sure to pass and sure to only read
- Answer every tool request, even a broken one, with a result the model can read
- When you stop it, keep what it had already said and done
- Keep a record written before each step, so what was in flight is reported as *unknown*

What resilience never does

- **Context:** it reads the record context already keeps; it adds no store of its own
- **Control flow:** same loop, same stops; a retry is inside one call or one command, and the only new way out is `Down`
- **Input:** untouched, apart from one question at startup, asked with the same `read()`
- **What the model sees:** only what it's told about its own work: interrupted, cut off, partial, and the partials ESC used to throw away

What's missing: alive, but not fast or cheap

- Every call re-sends the whole conversation; only the system prompt is cached
- A command that prints a megabyte puts a megabyte in the next request
- Three commands that could run side by side wait for each other
- The summary is written by the same big model as everything else

Making each step smaller, faster and cheaper is its own layer.

Next: Performance

Lesson 9